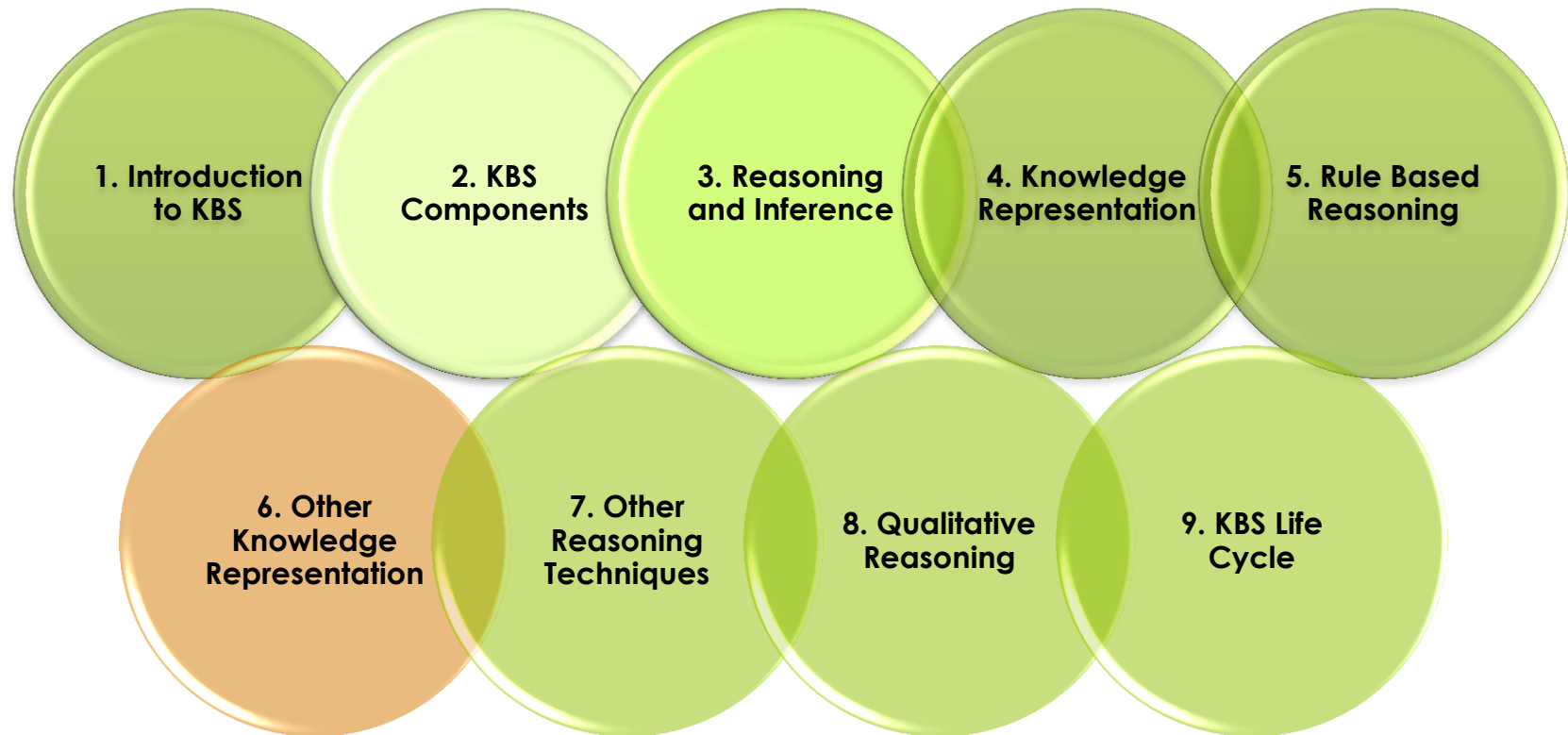


4/18/2016

BITI 2213: Knowledge Based System

Other Knowledge Representation

Course Overview



Knowledge Based System

Contents for Today

Other Knowledge Representations

1. Semantic Networks
2. Frames
3. Advantages and Disadvantages

Problems with Production Rules

- Simple implementations are **very inefficient**.
- Some types of knowledge are **not easily expressed** in production rules.
- Large sets of rules become **difficult to understand and maintain**.

SEMANTIC NETWORK

Semantic Nets

- A semantic net(work) is a graphical representation for **propositional** information.
- Is also called a **propositional** net.
- Propositions are forms of declarative knowledge because they state facts.
- A semantic net is a labeled, directed graph.
- Developed by **M. R. Quillian** for AI as a way of representing human memory and language understanding.
- Quillian used semantic nets to analyze the meanings of words in sentences.
- Since then semantic nets have been applied to many problems involving knowledge representation.

Semantic Nets

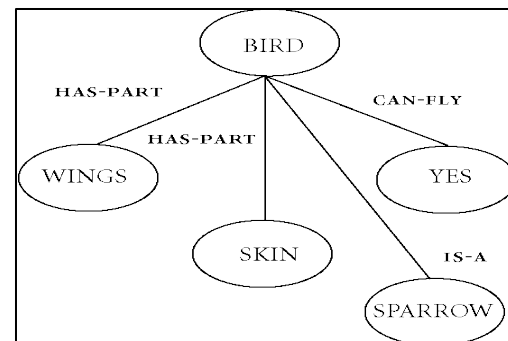
- The idea behind a semantic network is that knowledge is often best understood **as a set of concepts that are related to one another.**

The meaning of a concept is defined by its relationship to other concepts.

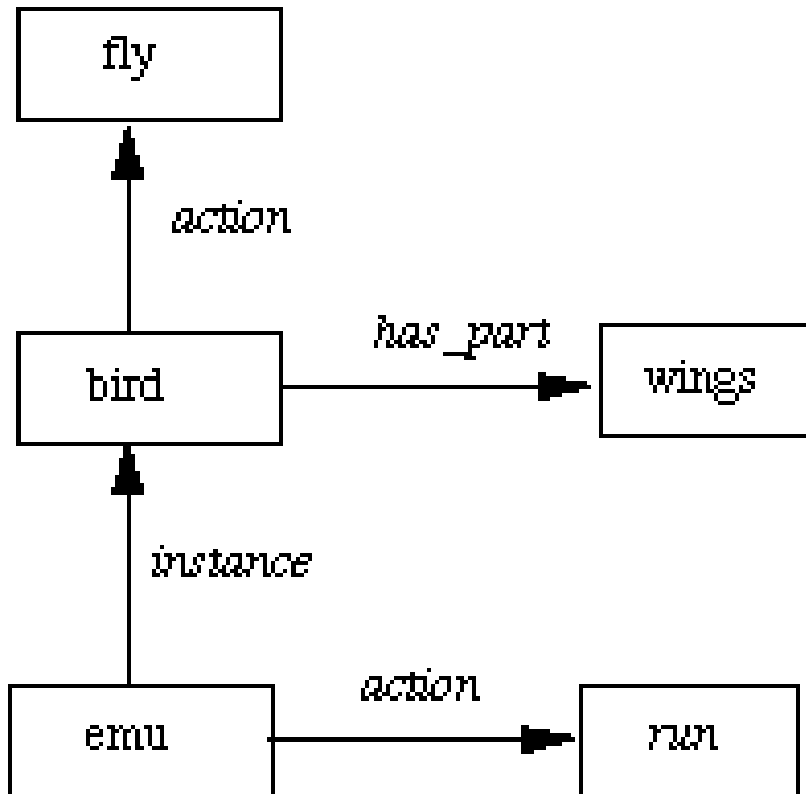
- A semantic network consists of a set of **nodes** that are connected by **labeled arcs**. The **nodes represent concepts** and the **arcs represent relations between concepts.**

Semantic Nets: Structure

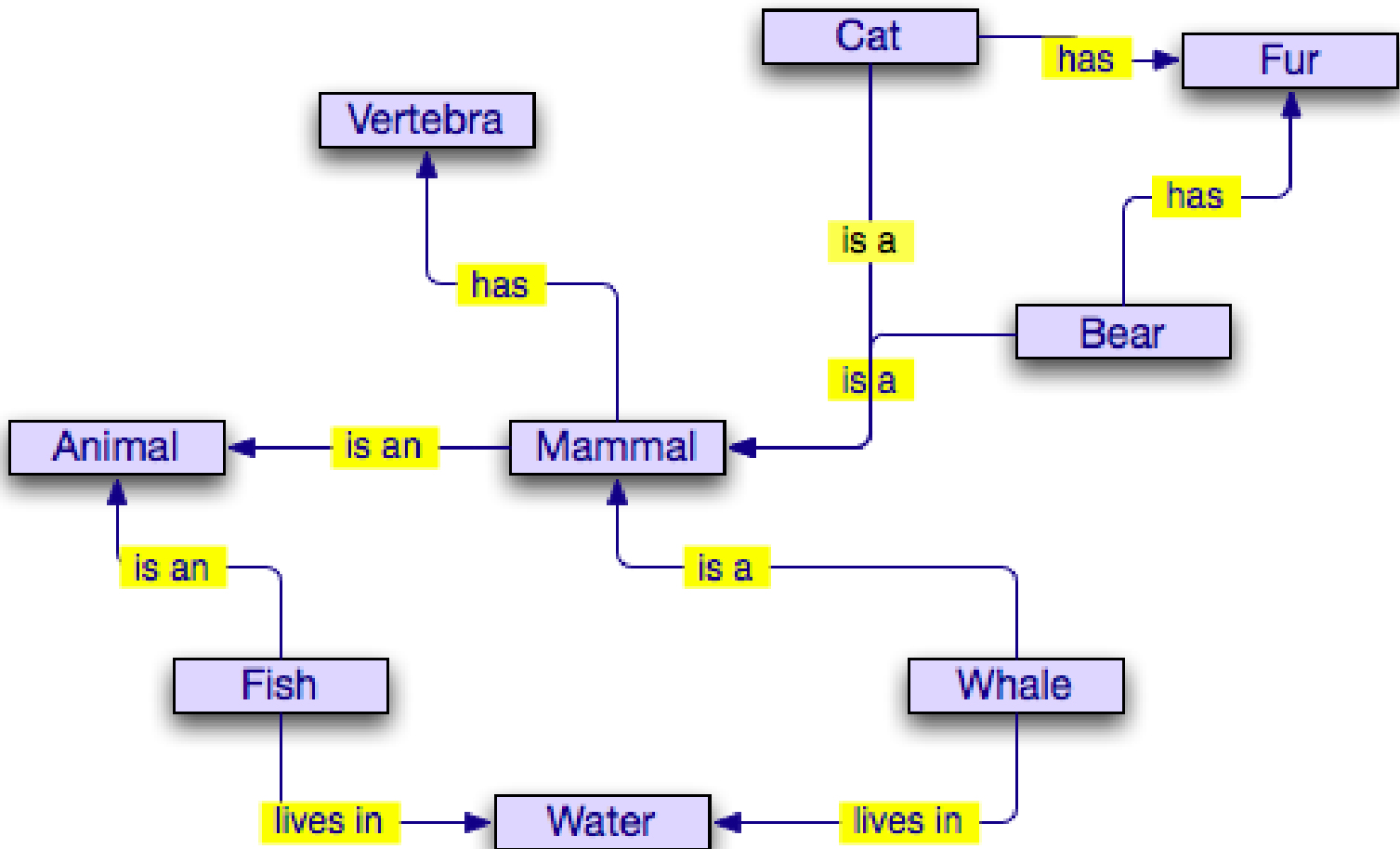
- Nodes represent objects, concepts, or situations
 - labels indicate the name
 - nodes can be **instances** (individual objects) or **classes** (generic nodes)
- Links represent relationships
 - the relationships contain the structural information of the knowledge to be represented
 - the label indicates the type of the relationship



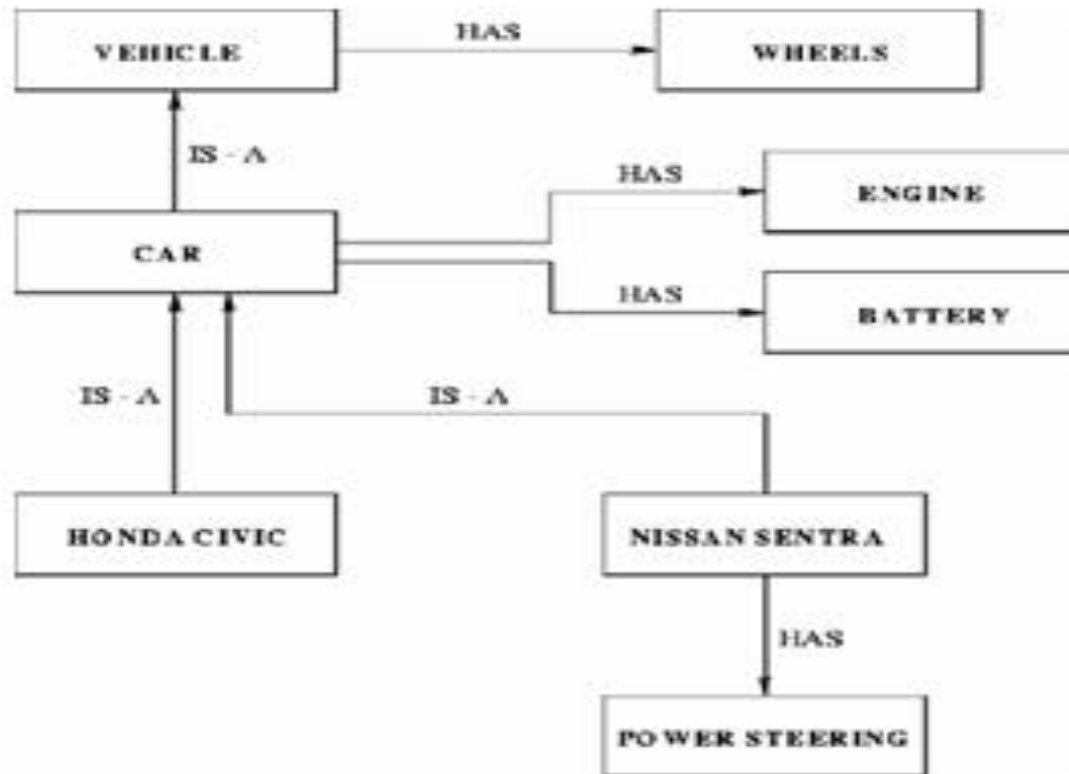
Semantic Nets: Example



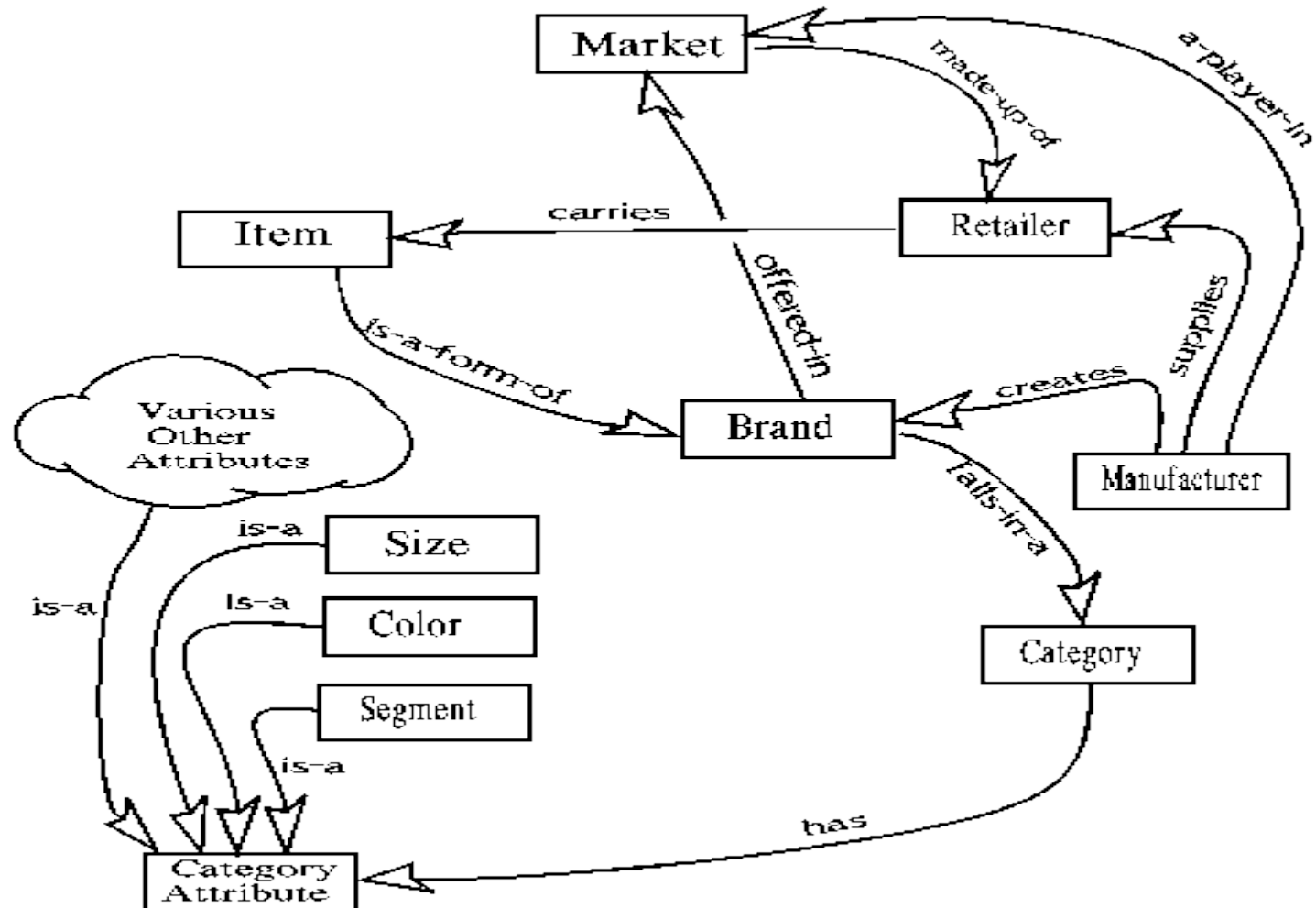
Semantic Net Example



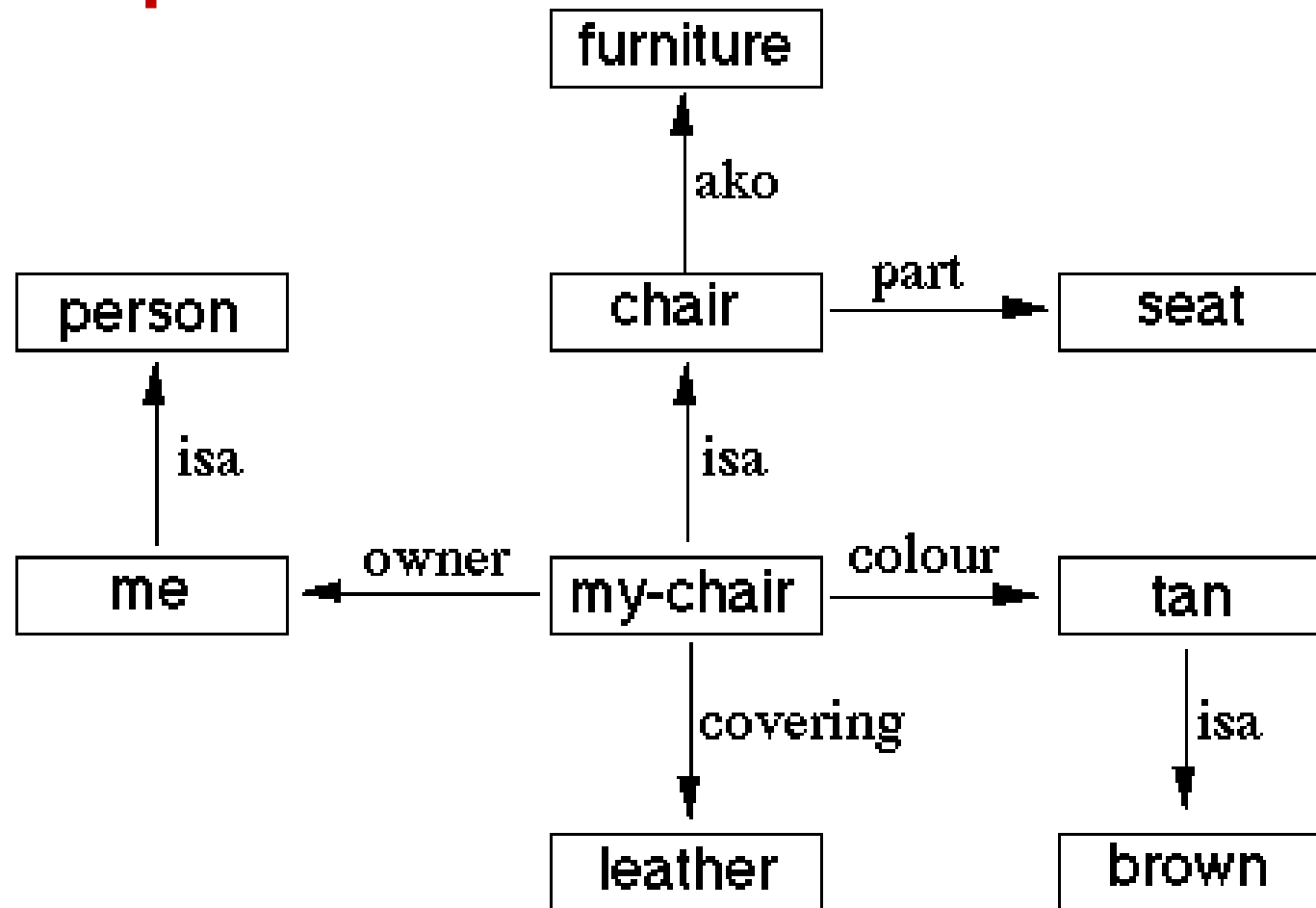
Semantic Nets: Example



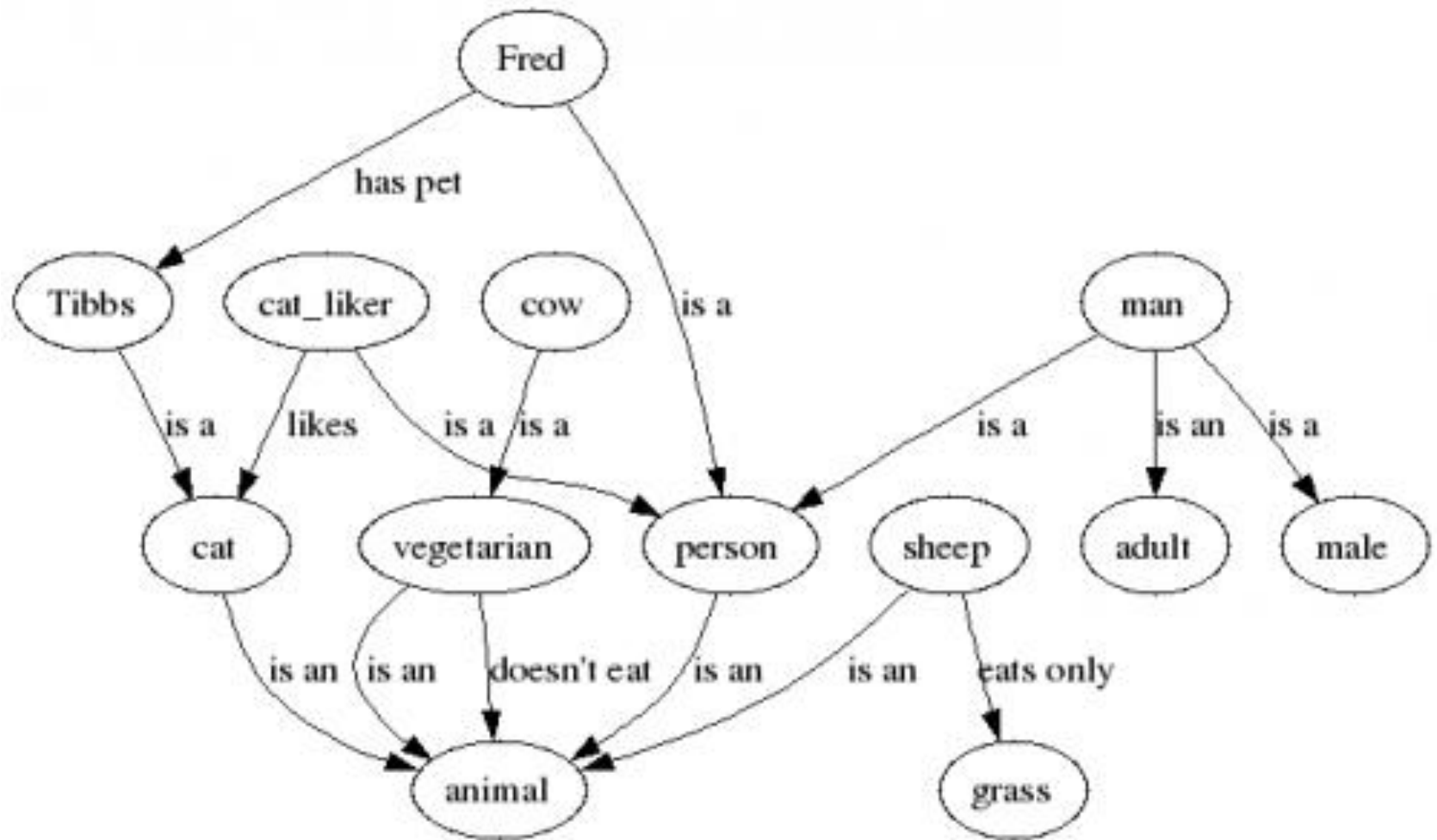
Semantic Nets: Example



Semantic Nets: Example

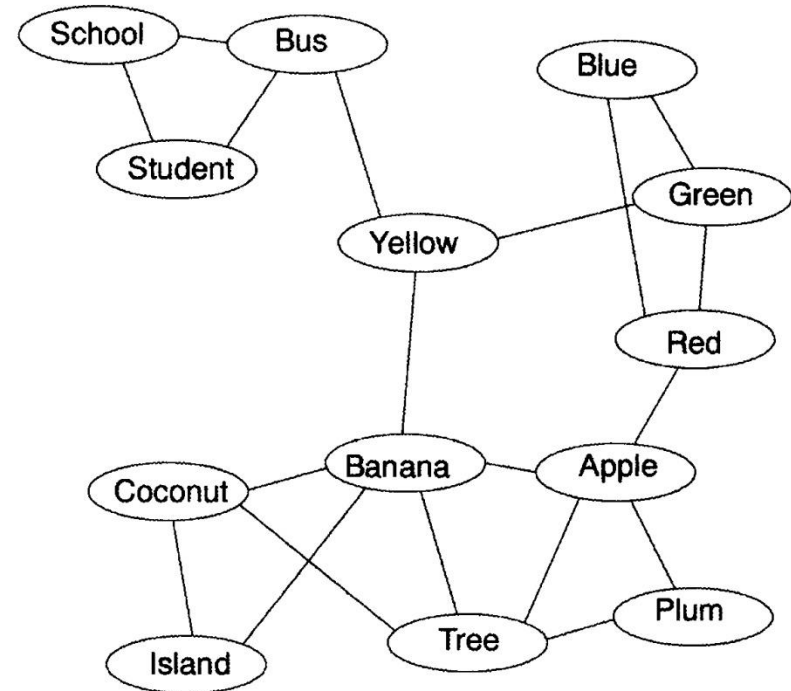


Semantic Nets: Example

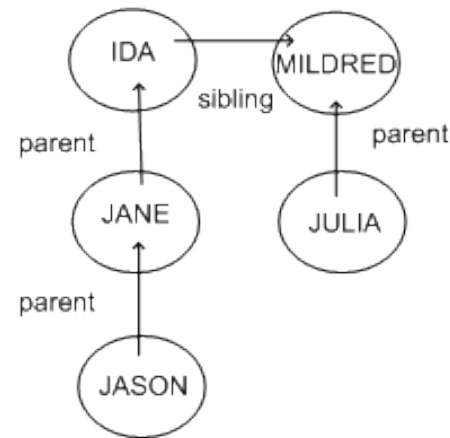
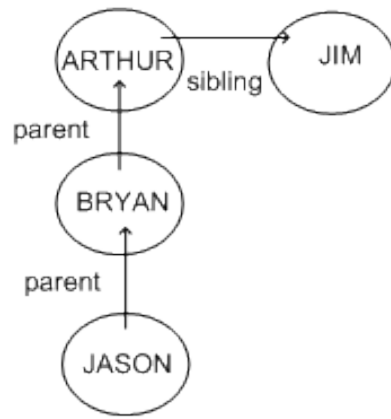


Relationships

- Without relationships, knowledge is **an unrelated collection** of facts.
- reasoning about these facts is not very interesting: inductive reasoning is possible



Relationships



- Relationships **express structure in the collection of facts.**
 - this allows the generation of meaningful new knowledge
 - generation of new facts
 - generation of new relationships

Types of Relationships

- Relationships can be arbitrarily defined by the knowledge engineer
 - allows great flexibility
 - for reasoning, the inference mechanism must know how relationships can be used to generate new knowledge
 - inference methods may have to be specified for every relationship

Types of Relationships

- Frequently used relationships (Winston 84)
 - **IS-A**
 - relates an instance (individual node) to a class (generic node)
 - **AKO** (a-kind-of)
 - relates one class (subclass) to another class (superclass)

Types of Relationships

INSTANCE: X is an *INSTANCE* of Y if X is a specific example of the general concept Y.

Example: *Elvis* is an **INSTANCE** of *Human*

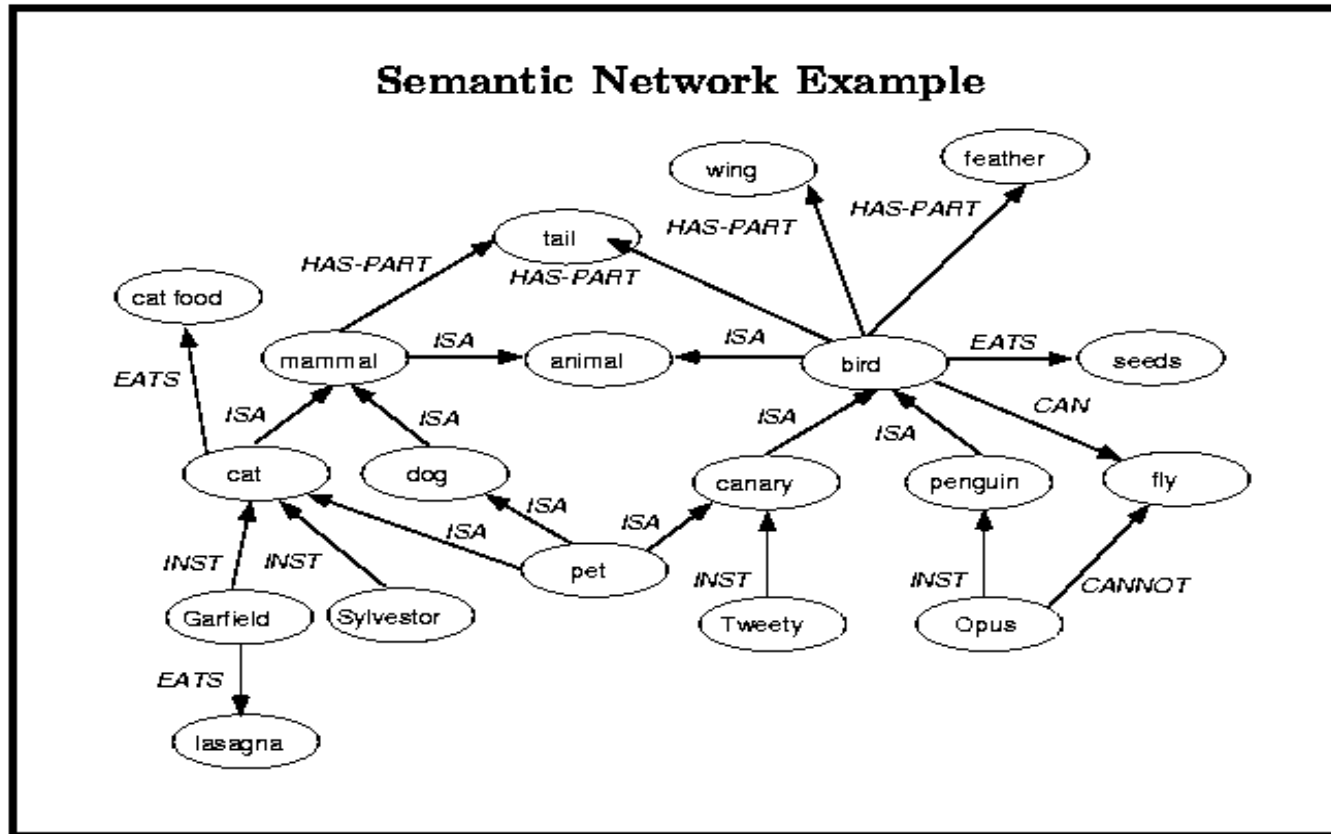
ISA: X *ISA* Y if X is a subset of the more general concept Y.

Example: *sparrow* **ISA** *bird*

HASPART: X *HASPART* Y if the concept Y is a part of the concept X. (Or this can be any other property)

Example: *sparrow* **HASPART** *tail*

Types of Relationships



Object and Attributes

- Attributes provide more detailed information on nodes in a semantic network
 - often expressed as *properties*
 - combination of attribute and value
 - attributes can be expressed as relationships
 - e.g. has-attribute

OAV-Triples

- object-attribute-value triplets
 - can be used to characterize the knowledge in a semantic net.
 - are convenient for listing knowledge in the form of table and translating the table into computer code by rule induction.
 - are useful for representing facts and the patterns to match the facts in the antecedent of a rule.

Object	Attribute	Value
apple	colour	red
apple	type	iphone
apple	quantity	100

Inheritance

- Inheritance is a key concept in semantic networks and can be represented naturally by following ISA links.
- In general, if concept X has property P, then all concepts that are a subset of X should also have property P.
- But exceptions are pervasive in the real world!
- In practice, inherited properties are usually treated as *default values*. If a node has a direct link that contradicts an inherited property, then the default is overridden.

Multiple Inheritance

- *Multiple inheritance* allows an object to inherit properties from multiple concepts.
- Multiple inheritance can sometimes allow an object to inherit conflicting properties.
- Conflicts are potentially unavoidable, so conflict resolution strategies are needed.

Representing Event

“Jack kidnapped Billy on August 5”

Kidnapping Event

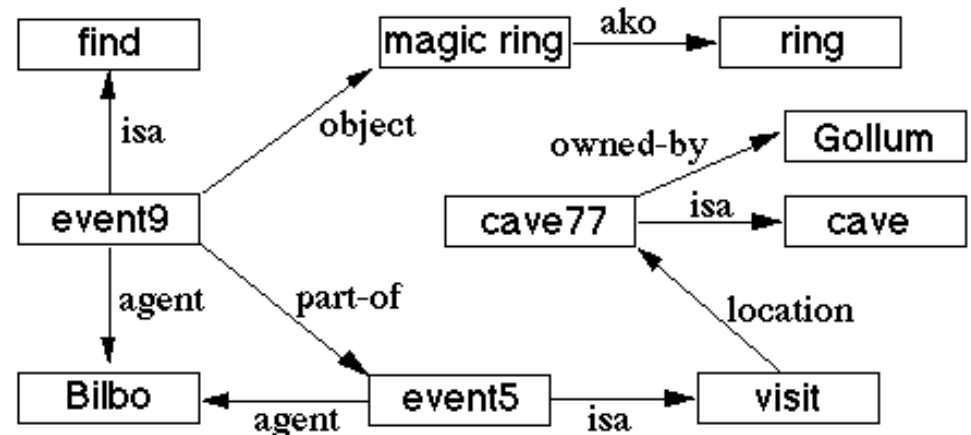
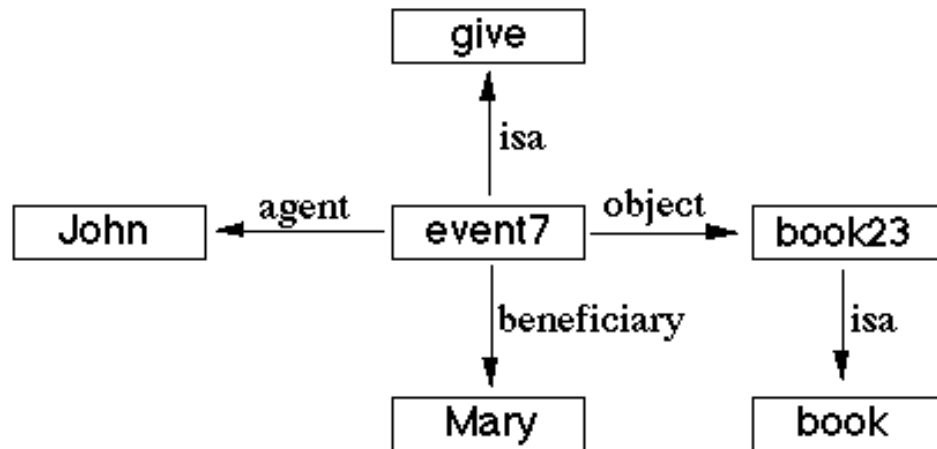
Kidnap1

perpetrator: Jack

victim: Billy

date: August 5

Semantic Net Example



Representing Predicates

score(Mavs, Bulls, 120-101)

Game

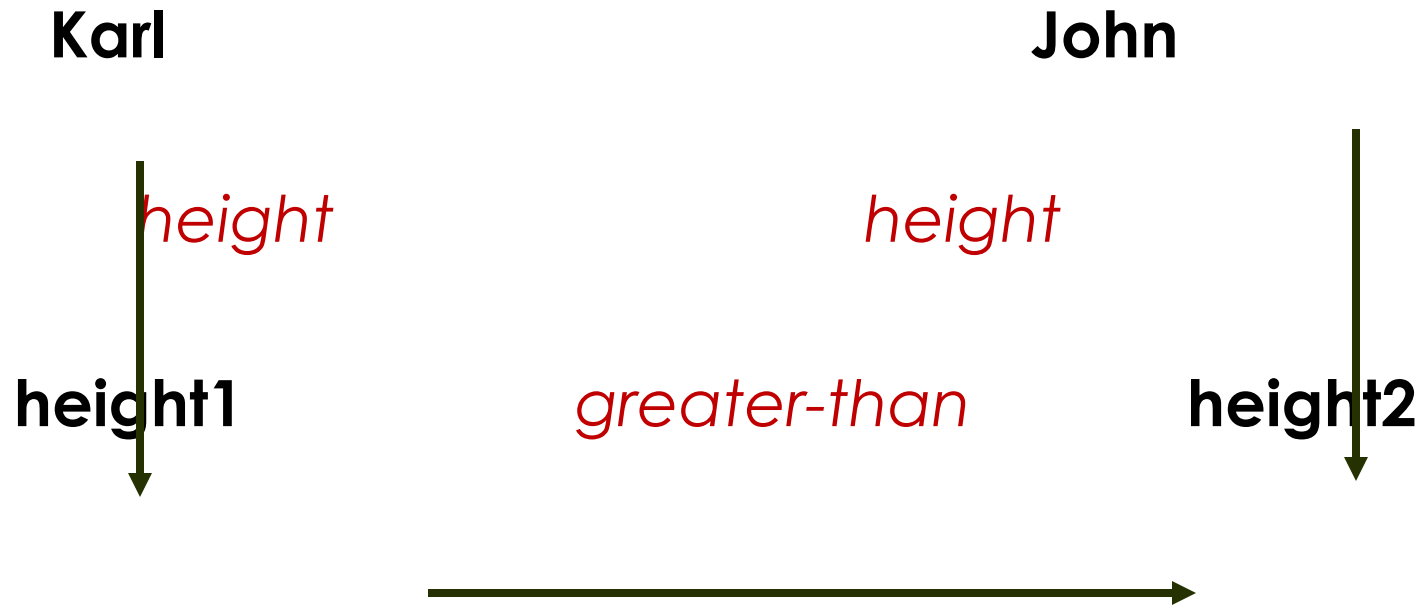
Instance: Game17

Hometeam: Mavs

Visiting team: Bulls

Score: 120-121

Representing Relations

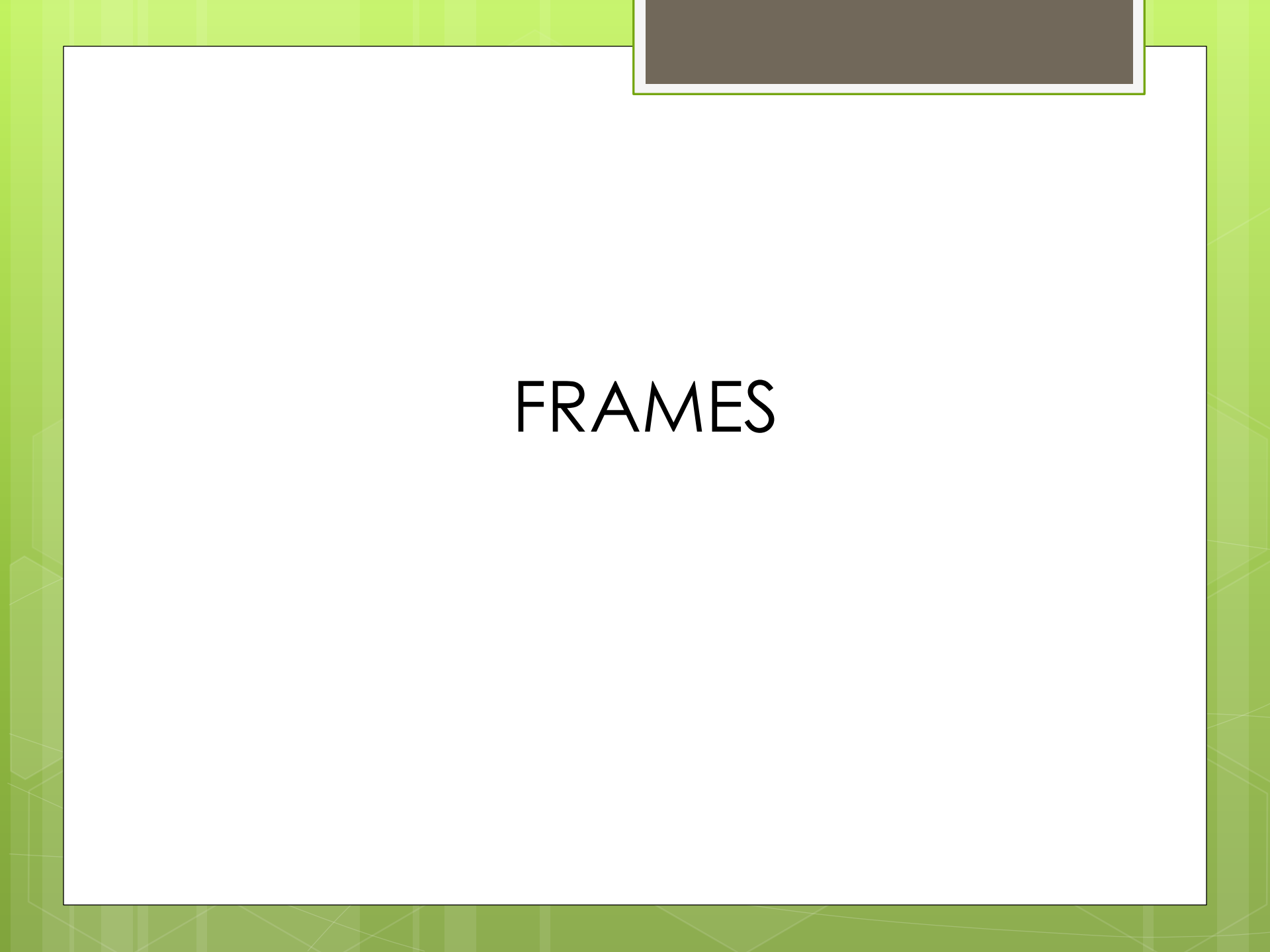


Implementation Questions

- Simple and efficient representation schemes for semantic nets
 - tables that list all objects and their properties
 - tables or linked lists for relationships
- Conversion into different representation methods
 - predicate logic
 - nodes correspond variables or constants
 - links correspond to predicates
 - propositional logic
 - nodes and links have to be translated into propositional variables and properly combined with logical connectives

Problems Semantic Nets

- Expressiveness
 - no internal structure of nodes
 - relationships between multiple nodes
 - no easy way to represent heuristic information
 - extensions are possible, but cumbersome
 - best suited for binary relationships
- Efficiency
 - may result in large sets of nodes and links
 - search may lead to combinatorial explosion
 - especially for queries with negative results
- Usability
 - lack of standards for link types
 - naming of nodes
 - classes, instances



FRAMES

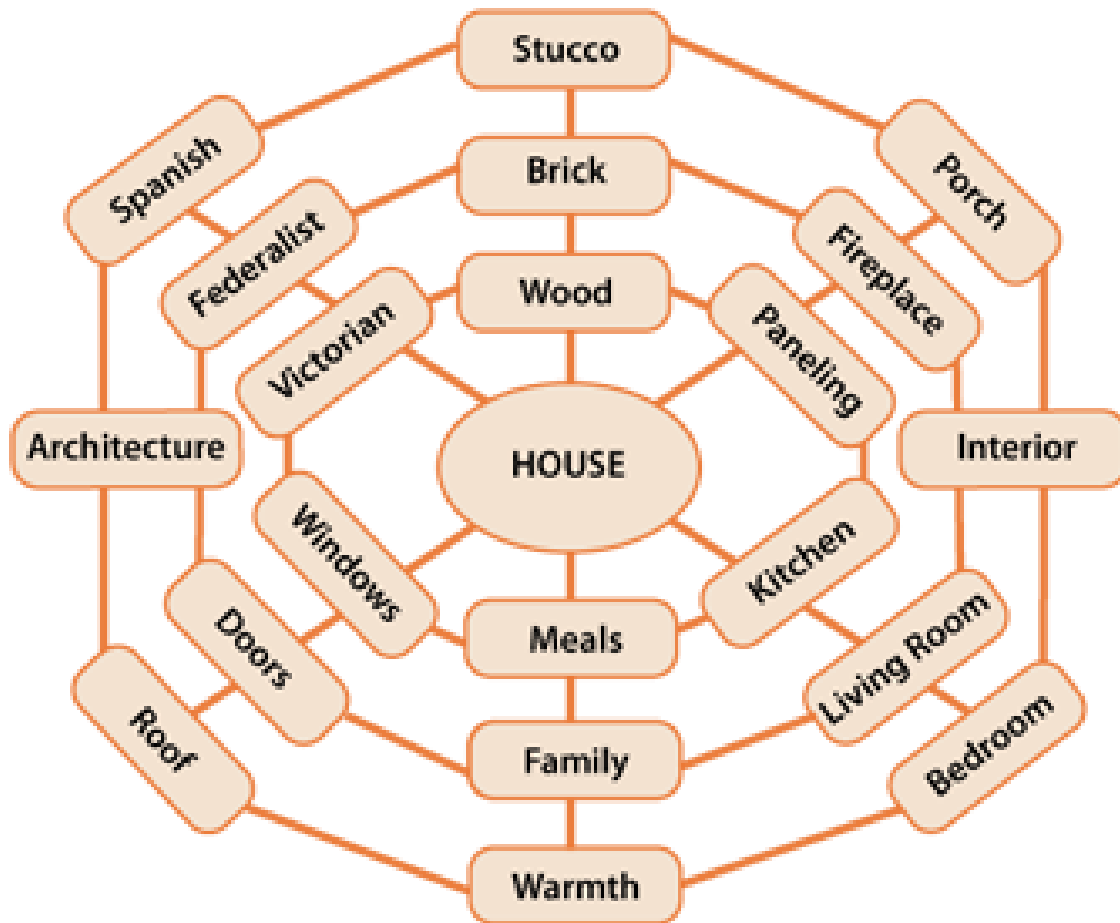
Schemata

- Suitable for the representation of more complex knowledge
 - causal relationships between a percept or action and its outcome
 - “deeper” knowledge than semantic networks
 - nodes can have an internal structure
 - for humans often tacit knowledge
- Related to the notion of *records* in computer science

Concept Schemata

- Abstraction that captures general/typical properties of objects
 - has the most important properties that one usually associates with an object of that type
 - may be dependent on task, context, background and capabilities of the user, ...
 - similar to stereotypes

Concept Schemata



Schemata Example

- The most frequently used instances of schemata are:
 - frames [Minsky 1975]
 - scripts [Schank 1977]
- Frames consist of a group of slots and fillers to define a stereotypical objects
- Scripts are time-ordered sequences of frames

Frames

- A frame represents an entity as a set of slots (attributes) and associated values.
- Each slot may have constraints that describe legal values that the slot can take.
- A frame can represent a specific entity, or a general concept.
- Frames are implicitly associated with one another because the value of a slot can be another frame.

Frames: Example

<i>Slot Name</i>	<i>Filler</i>
name	Osimo
height	small
weight	low
profession	warrior
armor	helmet
intelligence	very high
marital status	presumed single

Frames: Example

Novel	
Number of pages	Many pages
Cost	10 pounds
Format	Textual
Length of time to write	Very long time
Length of time to read	8 hours

← concept

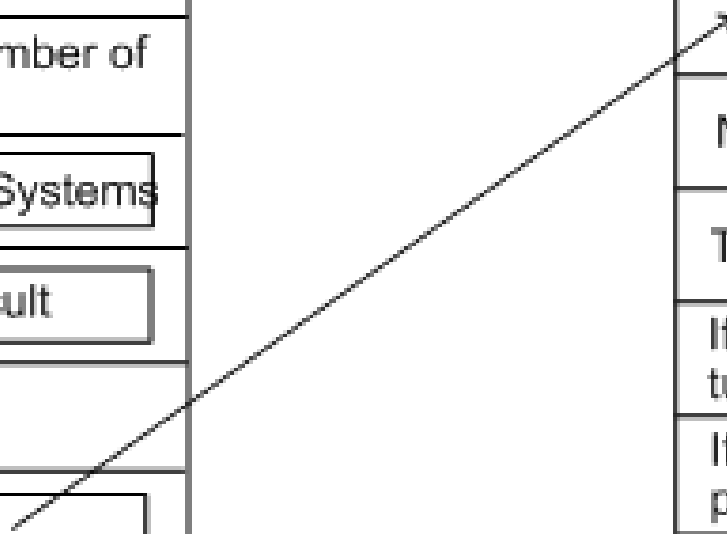
attributes

values

Frames: Example

Lecture	
Specialisation of: meeting	
Context: large number of students	
Course:	Op. Systems
Level:	Difficult
If difficult, then pay attention	
Lecturer:	
Room*:	

Lecturer	
Name:	Prof Jones
Tolerance:	Intolerant
If intolerant, then turn off mobile phone	
If intolerant, then pay attention	



Mammal

isa: ANIMAL

*haspart: HAIR

*breathes: AIR

NBA_BASKETBALL_PLAYER

isa: ADULTMALE

cardinality: 400

*height: > 6'

*salary: > \$200,000

HUMAN

isa: MAMMAL

cardinality: 6 million

*haspart: LEGS(2)

MICHAELJORDAN

instance: NBABASKETBALLPLAYER

height: 6'9"

ADULTMALE

isa: HUMAN

cardinality: 2 million

*gender: male

JOHNSTOCKTON

instance: NBABASKETBALLPLAYER

height: 6'1"

An asterisk (*) means that the slot can be inherited.

Overview of Frame Structure

- Two basic elements: *slots* and *facets* (fillers, values, etc.);
- Typically have parent and offspring slots
 - used to establish a property inheritance hierarchy (e.g., specialization-of)
- Descriptive slots
 - contain declarative information or data (static knowledge)

Overview of Frame Structure

- Procedural attachments (**demons**)
 - contain functions which can direct the reasoning process (dynamic knowledge)
(e.g., "activate a certain rule if a value exceeds a given level")
- Data-driven, event-driven (bottom-up reasoning)
- Expectation-drive or top-down reasoning
- Pointers to related frames/scripts - can be used to transfer control to a more appropriate frame

Slots

- Each slot contains one or more facets
- Facets may take the following forms:
 - values
 - default
 - used if there is not other value present
 - range
 - what kind of information can appear in the slot
 - if-added
 - procedural attachment which specifies an action to be taken when a value in the slot is added or modified (data-driven, event-driven or bottom-up reasoning)
 - if-needed
 - procedural attachment which triggers a procedure which goes out to get information which the slot doesn't have (expectation-driven; top-down reasoning)
 - other
 - may contain frames, rules, semantic networks, or other types of knowledge

Usage of Frames

- Filling slots in frames
 - can inherit the value directly
 - can get a default value.
 - these two are relatively inexpensive.
 - can derive information through the attached procedures (or demons) that also take advantage of current context (slot-specific heuristics).
 - filling in slots also confirms that frame or script is appropriate for this particular situation.

Demons

- One of the main advantages of frames is the ability to include demons to compute slot values.
- A demon is a function that computes the value of a slot on demand.

HUMAN

isa: (MAMMAL)

mortal: (yes :inheritable yes)

cardinality: (6 million :inheritable no)

age: (:inheritable yes :demon computes age)

MARY

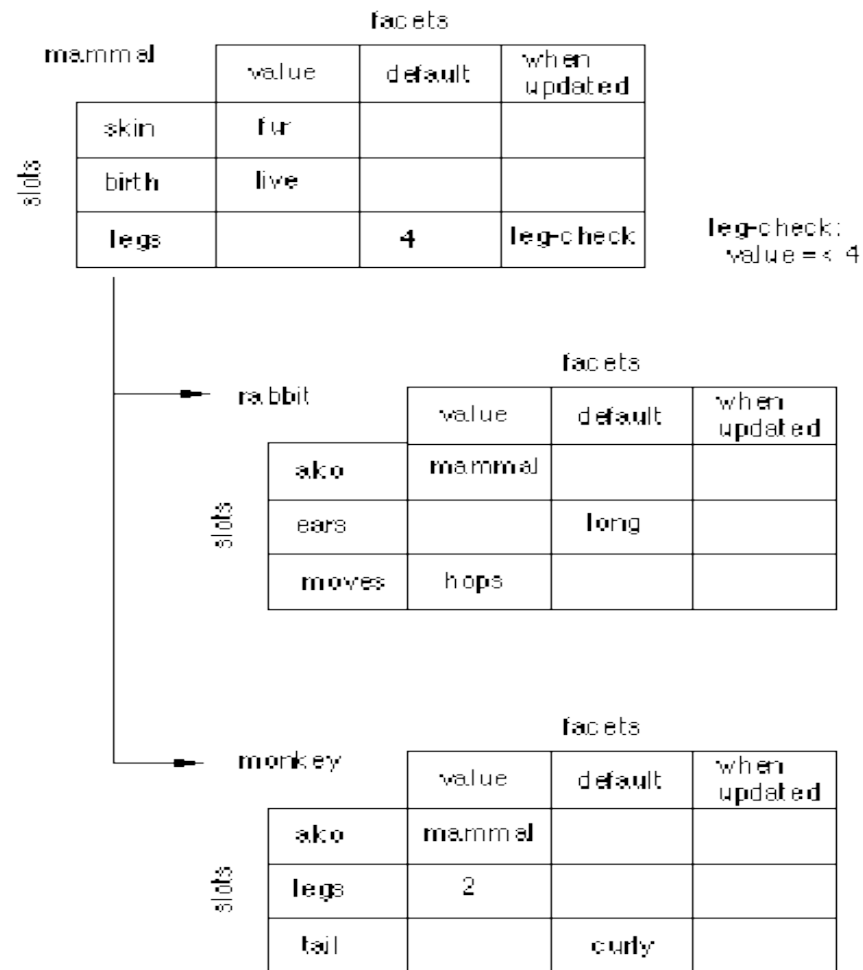
instance: HUMAN

gender: FEMALE

birthday: 11/04/90

```
int Compute_Age (frame)
return(today- (query birthday slot));
```

A Frame Example



Restaurant Frame Example

- generic template for restaurants
 - different types
 - default values
- script for a typical sequence of activities at a restaurant

Generic RESTAURANT Frame

Specialization-of: Business-Establishment

Types:

range: (Cafeteria, Fast-Food, Seat-Yourself, Wait-To-Be-Seated)

default: Seat-Yourself

if-needed: IF plastic-orange-counter THEN Fast-Food,
IF stack-of-trays THEN Cafeteria,
IF wait-for-waitress-sign or reservations-made THEN Wait-To-Be-Seated,
OTHERWISE Seat-Yourself.

Location:

range: an ADDRESS

if-needed: (Look at the MENU)

Name:

if-needed: (Look at the MENU)

Food-Style:

range: (Burgers, Chinese, American, Seafood, French)

default: American

if-added: (Update Alternatives of Restaurant)

Times-of-Operation:

range: a Time-of-Day

default: open evenings except Mondays

Payment-Form:

range: (Cash, CreditCard, Check, Washing-Dishes-Script)

Event-Sequence:

default: Eat-at-Restaurant Script

Alternatives:

range: all restaurants with same Foodstyle

if-needed: (Find all Restaurants with the same Foodstyle)

EAT-AT-RESTAURANT Script

Props: (Restaurant, Money, Food, Menu, Tables, Chairs)

Roles: (Hungry-Persons, Wait-Persons, Chef-Persons)

Point-of-View: Hungry-Persons

Time-of-Occurrence: (Times-of-Operation of Restaurant)

Place-of-Occurrence: (Location of Restaurant)

Event-Sequence:

first: Enter-Restaurant Script

then: if (Wait-To-Be-Seated-Sign or Reservations)
then Get-Maitre-d's-Attention Script

then: Please-Be-Seated Script

then: Order-Food-Script

then: Eat-Food-Script unless (Long-Wait) when Exit-Restaurant-Angry Script

then: if (Food-Quality was better than Palatable)
then Compliments-To-The-Chef Script

then: Pay-For-It-Script

finally: Leave-Restaurant Script

Frame Advantages

- Fairly intuitive for many applications
 - similar to human knowledge organization
 - suitable for causal knowledge
 - easier to understand than logic or rules
- Very flexible

Frame Problems

- It is tempting to use frames as *definitions* of concepts
 - not appropriate because there may be valid instances of a concept that do not fit the stereotype
 - *exceptions* can be used to overcome this
 - can get very messy
- Inheritance
 - not all properties of a class stereotype should be propagated to subclasses
 - alteration of slots can have unintended consequences in subclasses

KR Languages and Programming Languages

- How is a knowledge representation language different from a programming language (e.g. Java, C++)?
 - programming languages can be used to express facts and states
- What about "there is a pit in [2,2] or [3,1] (but we don't know for sure)" or "there is a wumpus in some square"
- Programming languages are not expressive enough for situations with incomplete information
 - we only know some possibilities which exist

KR Languages and Programming Languages

- How is a knowledge representation language different from natural language?
 - e.g. English, Spanish, German, ...
- Natural languages are expressive, but have evolved to meet the needs of communication, rather than representation
- The meaning of a sentence depends on the sentence itself and on the context in which the sentence was spoken, e.g. "Look!"
- Sharing of knowledge is done without explicit representation of the knowledge itself
- Ambiguous (e.g. small dogs and cats)

Good Knowledge Representation Languages

- Combines the best of natural and formal languages:
 - expressive
 - concise
 - unambiguous
 - independent of context
 - what you say today will still be interpretable tomorrow
- efficient
 - the knowledge can be represented in a format that is suitable for computers
 - practical inference procedures exist for the chosen format
- effective
 - there is an inference procedure which can act on it to make new sentences

Comparative Summary

- **Rules** are appropriate for some types of knowledge, but do not easily map to others.
- **Semantic nets** can easily represent inheritance and exceptions, but are not well-suited for representing negation, disjunction, preferences, conditionals, and cause/effect relationships.
- **Frames** allow arbitrary functions (demons) and typed inheritance. Implementation is a bit more cumbersome.

Summary Knowledge Representations

- Knowledge representation is very important for knowledge-based system
- Popular knowledge representation schemes are
 - rules, semantic nets, schemata (frames, scripts), logic
- The selected knowledge representation scheme should have appropriate inference methods to allow reasoning
- A balance must be found between
 - effective representation, efficiency, understandability